

Visor V2X en tiempo real

Observación en vivo del intercambio CAM/CPM/DENM sobre el gemelo digital,
cámara de seguimiento, HUD de percepción y replay mejorado

Universidad de Cuenca — Redes Vehiculares y Heterogéneas (INGE-00104)

Prof. Pablo Barbecho — 29 de agosto de 2026

Contenido

1	Objetivo	1
2	Marco teórico	2
2.1	Tres relojes: simulación, pared y render	2
2.2	Interpolación con retardo y <i>jitter</i> de red	3
2.3	La cadena en vivo: de la antena virtual al navegador	3
2.4	Reutilizar software libre: patrones frente a código	3
3	Requisitos previos	4
4	Parte 1 — Mensajes V2X en vivo	4
5	Parte 2 — Cámara cockpit (seguir a un vehículo)	5
6	Parte 3 — HUD de detección y filtro Emisor	5
7	Parte 4 — Modos sensor	5
8	Parte 5 — Replay mejorado	5
9	Casos reales de depuración	6
10	Ejercicios	7
11	Entregables	8

1 Objetivo

En la práctica de *Replay* el intercambio de mensajes V2X se analizó *a posteriori*, sobre los `.pcap` de una corrida ya terminada. En esta práctica el visor da un paso más: los mensajes se dibujan **mientras la simulación corre**, sobre los mismos vehículos que ns-3 está moviendo en ese instante. Además se incorporan herramientas de observación tomadas del mundo de la visualización geoespacial: cámara de seguimiento tipo *cockpit*, HUD de detección al estilo de un sistema de percepción, modos de sensor (visión nocturna, térmica) y un reproductor de replay con control fino de velocidad.

Al terminar, el estudiante podrá:

1. Observar y explicar el intercambio CAM/CPM/DENM *en vivo*, incluida la latencia de la cadena captura → disco → parser → web.

2. Distinguir **tiempo de simulación**, **tiempo de pared** y **cadencia de render**, y explicar quién “marca el reloj” en cada modo del visor.
3. Usar la cámara de seguimiento y el HUD de detección para razonar sobre qué percibe un nodo V2X.
4. Analizar seis casos reales de depuración del visor como ejemplos de fallos no triviales en sistemas distribuidos e interactivos.

Nota: prerequisite

Esta práctica asume completada la práctica “Análisis del intercambio de mensajes V2X: Replay web 3D vs. Wireshark”. Toda la teoría de la pila ITS-G5, CAM/DENM/CPM, ASN.1 UPER, RSSI y el modelo log-distancia está allí y no se repite aquí.

2 Marco teórico

2.1 Tres relojes: simulación, pared y render

En un visor de simulación conviven tres “tiempos” distintos:

- **Tiempo de simulación** (t_{sim}): el reloj interno de SUMO/ns-3. Avanza en pasos discretos (aquí 0,5 s por paso TraCI; los CAM se generan a 10 Hz, es decir cada 0,1 s simulado).
- **Tiempo de pared** (t_{wall}): el reloj del observador humano.
- **Cadencia de render**: cuántos cuadros por segundo dibuja el navegador (fps). No tiene por qué coincidir con ninguno de los dos anteriores.

La relación entre los tres define la *velocidad percibida*:

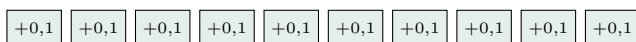
$$\text{velocidad} = \frac{\Delta t_{sim}}{\Delta t_{wall}} = \text{fps} \times \Delta t_{sim} \text{ por frame.}$$

Quién marca el reloj cambia según el modo. En vivo, el backend es el cliente TraCI #2 en *lockstep* con ns-3: no puede ir más rápido que la propia simulación (que a su vez depende de la CPU). En replay no hay simulador: el servidor decide libremente cuánto t_{sim} avanza por frame, y por eso puede reproducir a $0,25\times$ o a $4\times$. Esta asimetría es la clave del primer ejercicio.

Acoplado (mal): $10 \text{ fps} \times 0,5 \text{ s/frame} = 5 \text{ s simulados por segundo (5}\times\text{)}$



Desacoplado (bien): render fijo a 10 fps; avance simulado = $\frac{\text{mult}}{10}$ s/frame ($1\times \Rightarrow 0,1 \text{ s}$)



Cada caja = un frame enviado por el WebSocket durante 1 s de pared.

Figure 1: El replay original reproducía a $5\times$ sin quererlo: la cadencia de render (10 fps) estaba acoplada al paso de simulación (0,5 s). El arreglo desacopla ambos: se rinde siempre a 10 fps y el multiplicador del usuario decide cuánto tiempo simulado avanza cada frame.

2.2 Interpolación con retardo y *jitter* de red

El backend envía la posición de los vehículos ~ 2 veces por segundo; el navegador dibuja a 60 fps. Para que el movimiento sea continuo, el visor **interpola**: dibuja el pasado reciente, deslizando cada vehículo desde su posición anterior hacia la última recibida durante una *ventana* igual al período estimado entre frames.

El período real, sin embargo, no es constante: los frames llegan con *jitter* (variación de retardo). Si la ventana se estima con la *media* del período, por definición la mitad de los frames llegan *más tarde* que la media: el vehículo agota su ventana y se queda quieto esperando — el clásico movimiento “avanza-y-para”. La solución del visor es estirar la ventana un 30 % (`INTERP_STRETCH= 1,30`): se dibuja un poco más atrás en el tiempo, pero ya casi ningún frame llega “tarde”. Es la misma idea del *jitter buffer* de VoIP y videoconferencia: cambiar retardo por suavidad. La alternativa (extrapolar hacia el futuro) predice y se equivoca en cada giro; interpolar con retardo nunca inventa posiciones.

2.3 La cadena en vivo: de la antena virtual al navegador

En modo vivo los mensajes V2X no llegan al visor por un canal directo: ns-3 los escribe en sus `.pcap` y el backend los *relee del disco* mientras crecen. La cadena completa, con sus retardos, es:

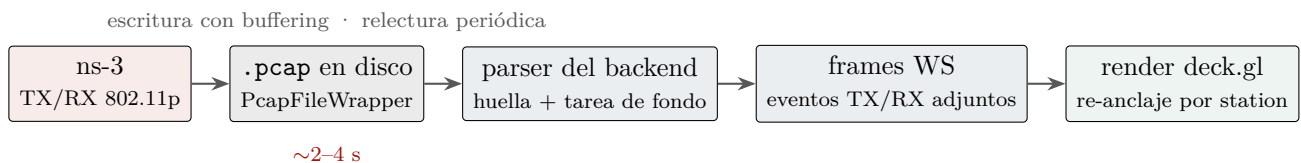


Figure 2: Cadena captura → disco → parser → WebSocket → render. El eslabón lento es el primero: ns-3 escribe los pcap con *buffering*, así que los arcos aparecen en el visor unos segundos después de la transmisión real. Es un retardo honesto del pipeline, no un error.

Dos decisiones de ingeniería sostienen esta cadena sin congelar el visor: el índice V2X se reconstruye en una **tarea de fondo** (nunca en el camino caliente del bucle de frames — ver el caso de depuración 5), y el refresco se *auto-regula*: se espera al menos $3\times$ la duración del último parseo antes de volver a parsear, de modo que un pcap cada vez más grande no acapare la CPU.

2.4 Reutilizar software libre: patrones frente a código

Varias de las funciones de esta práctica están inspiradas en **gods-eye-view**, un visor geoespacial open source (licencia MIT) con estética de “satélite de vigilancia” construido sobre CesiumJS. Integrarlo directamente era inviable — motor distinto (Cesium+Vite frente a MapLibre+deck.gl sin paso de build) — así que se portaron *patrones*, no código:

Patrón en gods-eye-view	Adaptación en SUMO-GEO
Cámara <i>cockpit</i> dentro de un avión	Cámara persecución de un vehículo SUMO
<i>Reskin reality</i> : shaders CRT/NVG/FLIR	Filtros CSS sobre el mapa + <i>scanlines</i>
Overlay de detección (cajas + IDs)	HUD: caja y etiqueta <code>veh<station></code>
“Render one interval behind”	Ventana de interpolación estirada (§2.2)
Iconos estables en pantalla	No aplicaba: nuestros vehículos ya son geometría 3D orientada por rumbo real

Nota de propiedad intelectual

La licencia MIT permite estudiar, adaptar y redistribuir el *código*, pero no cubre necesariamente los *assets* (modelos 3D, imágenes, datos) del proyecto, que pueden tener licencias propias. Portar ideas y patrones de diseño — como aquí — es siempre legítimo; copiar código exige respetar la licencia (atribución en MIT); copiar assets exige revisar su licencia una a una.

3 Requisitos previos

1. Práctica de instalación completada: stack `van3twin` operativo.
2. Práctica de Replay completada (teoría y flujo de pcaps ya conocidos).
3. El backend actualizado incluye el modo vivo y el control de velocidad del replay. Tras un `git pull` de `SUMO_GEO`, reconstruir **una vez**:

EN EL MAC (PC local)

```
cd ~/van3twin-docker
docker compose --profile visor build backend
docker compose --profile visor up -d
```

Recordatorio: nunca `docker compose build` a secas (reconstruiría también la imagen grande de `ns-3`). El frontend no necesita build: `nginx` sirve los estáticos con `no-store`; basta recargar el navegador.

4 Parte 1 — Mensajes V2X en vivo

Hasta ahora el visor en vivo mostraba *solo* la movilidad; los mensajes exigían esperar al replay. Ya no: el backend relee en caliente los pcap que `ns-3` escribe en su directorio de trabajo (el volumen ya está montado en `/ns3`; la ruta la fija `APP_LIVE_PCAP_DIR=/ns3/ns-3-dev`) y adjunta los eventos TX/RX a los frames en vivo.

DENTRO DEL CONTENEDOR `van3twin`

```
cd ~/Van3Twin/ns-3-dev
./ns3 run "v2v-emergencyVehicleAlert-80211p --sumo-gui=false
--met-sup=true --sim-time=300 --num-traci-clients=2"
```

SUMO queda “waiting for 2 clients”; el visor lo destraba al conectarse.

EN EL NAVEGADOR

Abrir <http://localhost:8081>. A los pocos segundos de arrancar la corrida:

- Brotan **pulsos** (transmisiones) y **arcos** (recepciones) sobre los vehículos en movimiento.
- El panel *Replay V2X* revela automáticamente “Mensajes V2X”, los filtros CAM/CPM/DENM y la fila **Emisor** (se puebla sola con las estaciones que van apareciendo; al elegir una, anillo ámbar sobre el emisor y solo sus mensajes en pantalla).
- El **panel PHY** muestra las estadísticas *de la corrida en curso* y se refresca cada 10 s: latencia, cobertura, PER y utilización van creciendo con la corrida.

Las etiquetas de los arcos muestran **metros** (distancia entre las posiciones TraCI en vivo de emisor y receptor) y **dBm** si el EVA parcheado está volcando **signal-rx.csv** en el mismo directorio.

Nota: el retardo es real

Los arcos aparecen ~2–4 s después de la transmisión: es el buffering de escritura de ns-3 más el ciclo de relectura (figura 2). La movilidad, en cambio, llega por TraCI sin pasar por disco: por eso los vehículos se mueven “al día” y los mensajes van unos segundos por detrás. Distinguir ambos caminos es parte del ejercicio 3.

5 Parte 2 — Cámara cockpit (seguir a un vehículo)

EN EL NAVEGADOR

1. Clic **derecho** sobre cualquier vehículo → en el popup, botón “*Seguir este vehículo*”.
2. La cámara se coloca en persecución: inclinación 76°, zoom 19.3, orientada al rumbo del vehículo. Un *badge* flotante muestra en vivo id, rumbo y km/h.
3. Salir: botón de cierre del badge, arrastrar el mapa o usar el pad.

Combinación útil: seguir a la **ambulancia** del escenario EVA con el filtro Emisor puesto en su station — se ve el DENM de emergencia saliendo del propio vehículo que la cámara persigue.

6 Parte 3 — HUD de detección y filtro Emisor

El checkbox *Detección* del panel principal activa un HUD estilo sistema de percepción: caja verde y etiqueta **veh<station>** sobre cada vehículo. Sirve para responder de un vistazo “¿qué está viendo/etiquetando la plataforma?” y para cruzar el station del HUD con el filtro Emisor y con los receptores listados en el popup de un mensaje.

Nota

El HUD comparte el límite de nivel de detalle del visor (LOD_MAX_DETAILED): con flotas muy grandes las cajas se apagan automáticamente, igual que el detalle 3D de los vehículos.

7 Parte 4 — Modos sensor

El selector de la barra superior cicla la “óptica” del visor: **Normal** / **NVG** (visión nocturna) / **FLIR** (térmica) / **Noir** / **CRT** (monitor antiguo con *scanlines* y viñeta). Técnicamente son filtros CSS aplicados al elemento del mapa — mapa y overlay deck.gl comparten elemento, así que vehículos y mensajes heredan el mismo tratamiento. Útiles para presentaciones y para las capturas de los informes.

8 Parte 5 — Replay mejorado

Con la corrida terminada, publicar los resultados y entrar en replay:

DENTRO DEL CONTENEDOR `van3twin`

```
cp v2v-EVA-*.pcap signal-rx.csv ~/results/
```

No hace falta reiniciar nada: el backend detecta los ficheros nuevos solo.

EN EL NAVEGADOR

Novedades respecto a la práctica anterior:

- **Barra de tiempo** estilo reproductor: progreso relleno y tiempos `m:ss` actual/total.
- **Velocidad** (fila tortuga-liebre): $0,25\times$ a $4\times$ sobre *tiempo real*. A $1\times$ un segundo de pared es un segundo simulado, con granularidad de 0,1 s por frame = un CAM a 10 Hz (§2.1).
- **El pulso de transmisión ES la cobertura**: el anillo se expande exactamente hasta el percentil 95 del alcance medido en la corrida (el valor *Cobertura p95* del panel PHY).
- Persistencia por tipo de evento: los arcos RX viven 2,6 s (tiempo de sobra para clicarlos e inspeccionar el ASN.1); los pulsos TX, 1,2 s.
- Pulsos y arcos van *anclados* a los vehículos dibujados: nacen y viajan con la posición interpolada, sin adelantarse.

9 Casos reales de depuración

Estas seis fallas aparecieron (y se corrigieron) durante el desarrollo de lo que acabas de usar. Cada una ilustra un concepto general; ninguna se veía en el código a simple vista. Léelas como mini-casos: síntoma → causa → arreglo → concepto.

CASO DE DEPURACIÓN 1: el vehículo que avanza y se para

Síntoma: movimiento a tirones pese a la interpolación. **Causa:** la ventana de interpolación usaba la *media* del período entre frames; con jitter, la mitad de los frames llegan más tarde que la media y el vehículo espera parado. **Arreglo:** ventana estirada 30 % (`INTERP_STRETCH`); cada frame reinicia desde la posición *ya dibujada*, así el error se autocorrigie sin saltos. **Concepto:** interpolación con retardo vs. extrapolación; jitter buffers (§2.2).

CASO DE DEPURACIÓN 2: el replay que iba a $5\times$

Síntoma: el replay “corría” aunque nadie tocó nada; en vivo no pasaba. **Causa:** $10\text{ fps} \times 0,5\text{ s/frame} = 5\text{ s simulados por segundo}$. En vivo no se nota porque ns-3, en lockstep, marca el ritmo. **Arreglo:** desacoplar render (10 fps fijos) de avance simulado (el comando `speed` del WebSocket acepta ahora el paso por frame). **Concepto:** los tres relojes de §2.1.

CASO DE DEPURACIÓN 3: “Seguir vehículo” se autocancelaba

Síntoma: la cámara persecución moría al instante de activarla. **Causa:** mover la cámara por código (`map.jumpTo`) dispara el mismo evento `rotatestart` que un gesto del usuario; el manejador “cancelar si el usuario rota” mataba el seguimiento en el primer frame de la propia cámara. **Arreglo:** distinguir gesto real de evento programático (`guard e.originalEvent`). **Concepto:** los sistemas de eventos no distinguen por defecto quién originó el evento; hay que preguntarlo.

CASO DE DEPURACIÓN 4: el botón inclickable

Síntoma: un botón dentro del popup ignoraba los clics. **Causa:** heredaba `pointer-events:none` del popup no interactivo. **Arreglo:** `pointer-events:auto` en el elemento. **Concepto:** las propiedades CSS heredadas o en cascada afectan a hijos que “no se ven” afectados; depurar con el inspector, no con el ojo.

CASO DE DEPURACIÓN 5: pausas periódicas en vivo

Síntoma: el stream en vivo se congelaba $\sim 0,5$ s cada ~ 2 s. **Causa:** el bucle de frames hacía `await` del re-parseo del índice V2X; como el pcap *crece continuamente*, la huella cambiaba en cada chequeo y el parseo (CPU-bound) bloqueaba el envío de frames. **Arreglo:** caché sin bloqueo + reconstrucción en tarea de fondo con throttle adaptativo ($3\times$ la duración real del último parseo). **Concepto:** en un event loop (asyncio) nunca se bloquea el camino caliente; el trabajo pesado va a hilos o tareas de fondo.

CASO DE DEPURACIÓN 6: arcos adelantados a sus vehículos

Síntoma: los arcos nacían “delante” del vehículo que los emitía. **Causa:** los arcos usaban la posición *cruda* del último frame; los vehículos dibujados van $\sim$ medio período por detrás (interpolación con retardo). Dos fuentes de verdad para la misma posición. **Arreglo:** re-anclar pulsos y arcos por station a la posición *interpolada* en cada tick de render. **Concepto:** cuando dos capas visualizan el mismo objeto, deben compartir la misma fuente de posición.

10 Ejercicios

TAREA

1. **¿Quién marca el reloj?** Reproduce la MISMA corrida en vivo y en replay. ¿Por qué el replay admite $4\times$ y el vivo no admite acelerarse? ¿Qué limitaría al vivo si quisiéramos acelerarlo? Responde usando los tres relojes de §2.1.
2. **La ambulancia.** Con cockpit + filtro Emisor, sigue a la ambulancia del EVA. Anota qué tipos de mensaje emite, con qué frecuencia aparecen sus pulsos y qué vehículos aparecen como receptores en el popup cuando pasa cerca de ellos.
3. **El retardo del vivo.** Cronometra cuánto tardan los arcos en aparecer tras arrancar la corrida. Explica el retardo eslabón por eslabón con la figura 2, y explica por qué la *movilidad* no sufre ese retardo.
4. **Cobertura visible.** En replay, mide a ojo (con las etiquetas de distancia de los arcos como regla) el radio final de un pulso TX y compáralo con el valor *Cobertura p95* del panel PHY. ¿Por qué se eligió el p95 y no el máximo?
5. **Path-loss otra vez.** Con las etiquetas “m · dBm” de al menos cinco arcos a distintas distancias, estima el exponente n de log-distancia (método de la práctica de Replay) y compáralo con el modelo configurado en ns-3 (LogDistance, $n = 3,0$).
6. **Depuración guiada (opcional, para valientes).** En `app.js`, comenta el `guard.e.originalEvent` del manejador de rotación y recarga. Activa “Seguir vehículo” y describe lo que ocurre y por qué (caso 3). Restaura el guard al terminar.
7. **Granularidad.** A $1\times$ el replay avanza 0,1 s simulados por frame. ¿Qué relación tiene

ese número con la frecuencia de generación de CAM? ¿Qué se “perdería” visualmente con 0,5 s por frame?

11 Entregables

Informe breve (PDF) con: respuestas razonadas a los ejercicios 1–5 y 7 (el 6 es opcional), capturas del visor que respalden cada respuesta (los modos sensor son bienvenidos para las capturas), y una reflexión final de un párrafo: ¿qué aporta ver los mensajes *en vivo* que no aporta el replay, y viceversa?

Universidad de Cuenca · Redes Vehiculares y Heterogéneas · Prácticas del gemelo digital VaN3Twin + SUMO-GEO. Prerrequisitos: *Práctica de instalación del framework* y *Práctica Replay V2X*.